ARCHITECTURE GUIDE

ErrorBuddy

A small compiler, written in C++17 using only the standard library. It turns a beginner language into three-address code (TAC) and explains every error in plain English. This guide shows which file does what and how a program flows through the compiler.

5
compiler phases in 3 C++ source files

~730
lines of C++ (lexer, parser, driver and headers)

10 / 10
example programs pass `make test` (checked 26 Oct 2026)

The project at a glance

FILE	LINES	ROLE
`main.cpp`	88	Driver. Reads the file, runs the phases in order, prints the result or the error, sets the exit code.
`lexer.h` / `lexer.cpp`	48 / 114	Phase 1. `Token`, `TokenType`, `tokenize()`. Also defines `CompileError`, the error type every phase uses.
`parser.h` / `parser.cpp`	80 / 396	Phases 2–4 in one pass: recursive descent, symbol table checks and TAC generation. Also holds most of Phase 5 (diagnosis and typo hints).
`Makefile`	45	`make`, `run`, `test`, `web`, `clean`.
`examples/`	10 + 10	Test programs and their expected output in `examples/expected/*.out`.
`web/server.py`, `web/index.html`	89 / 288	Browser playground. Runs the real binary and shows the result.
`case_study/`	972	Evaluation scripts: a corpus of 24 mistakes, the ErrorBuddy vs gcc comparison and the PDF builder.

The language it compiles

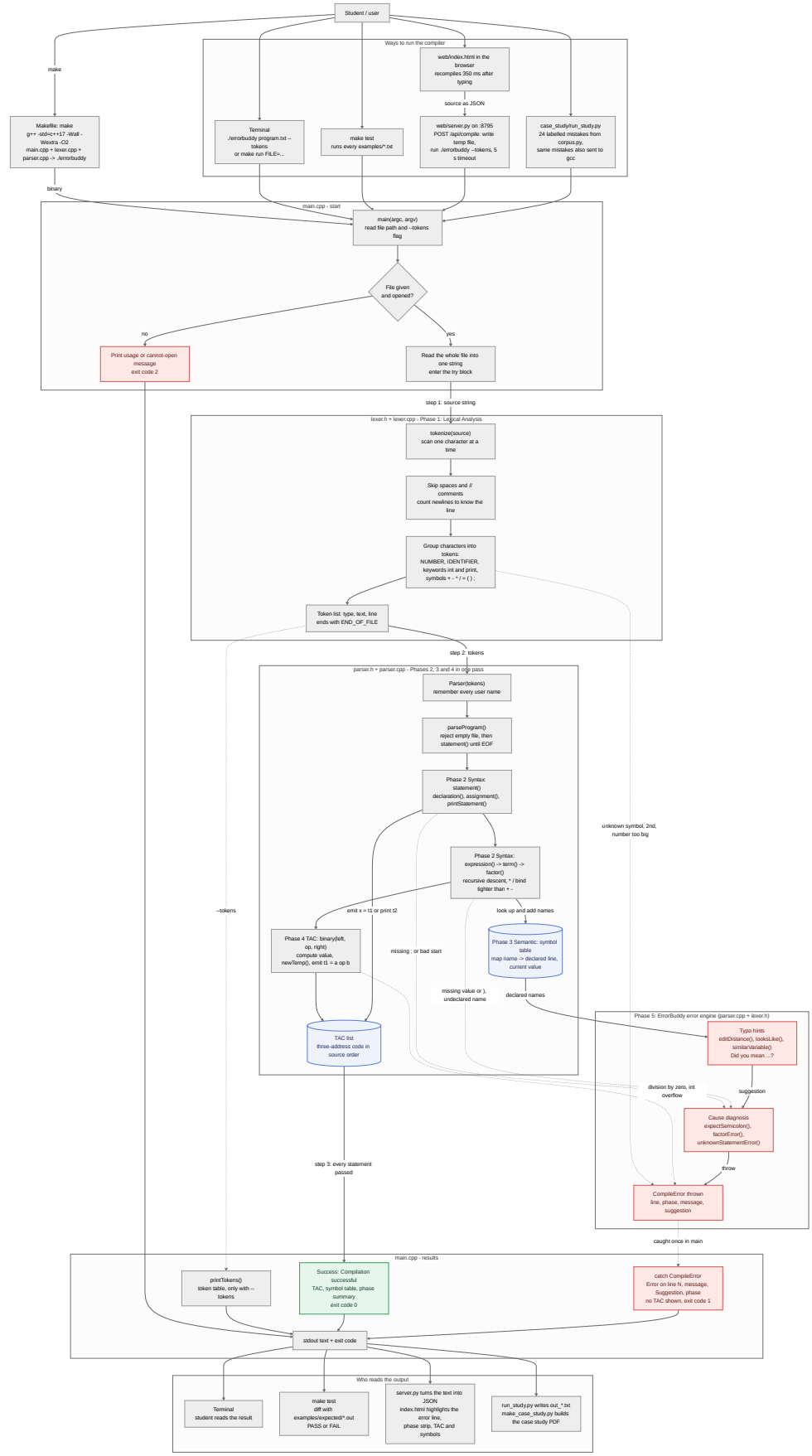
```
int age = 20;              // declare (value required)
int score = 10;
score = score + age * 2;   // assign
print (score - 5) / 3;     // print any expression
```

Every variable is an `int` and needs a starting value. Arithmetic is `+ - * /` with normal precedence and parentheses. Each statement ends with `;`, and `//` starts a comment. The language has no unary minus, loops, if/else or functions, and compilation stops at the first error.

Mermaid sources for Excalidraw. All four diagrams in this guide are in `~/ErrorBuddy/architecture/*.mmd`. In Excalidraw, open *More tools → Mermaid to Excalidraw*, paste one file's contents and click *Insert*. The source for the full diagram is also printed in the appendix.

DIAGRAM 1 · 1_FULL_ARCHITECTURE.MMD

Full architecture and execution flow



error path success data the parser builds Dashed arrows are throws or optional output.

RESPONSIBILITIES

What each file does

main.cpp: the driver

FUNCTION / BLOCK	WHAT IT DOES
`main(argc, argv)`	Reads the file path and the optional `--tokens` flag. With no path it prints usage and returns 2. If the file can't be opened it says so and returns 2.
`try` block	Calls `tokenize()`, then `Parser(tokens).parseProgram()`. Reaching the end means every phase passed: it prints the TAC, the symbol table and a phase summary, and returns 0.
`printTokens()`	With `--tokens`, prints the lexer output as a line / type / text table before parsing starts.
`catch (CompileError)`	Phase 5 output: *Error on line N*, the message, the suggestion and the phase that found it. No TAC is printed. Returns 1. Any other exception is reported as an internal error and returns 3.

lexer.h / lexer.cpp: Phase 1, lexical analysis

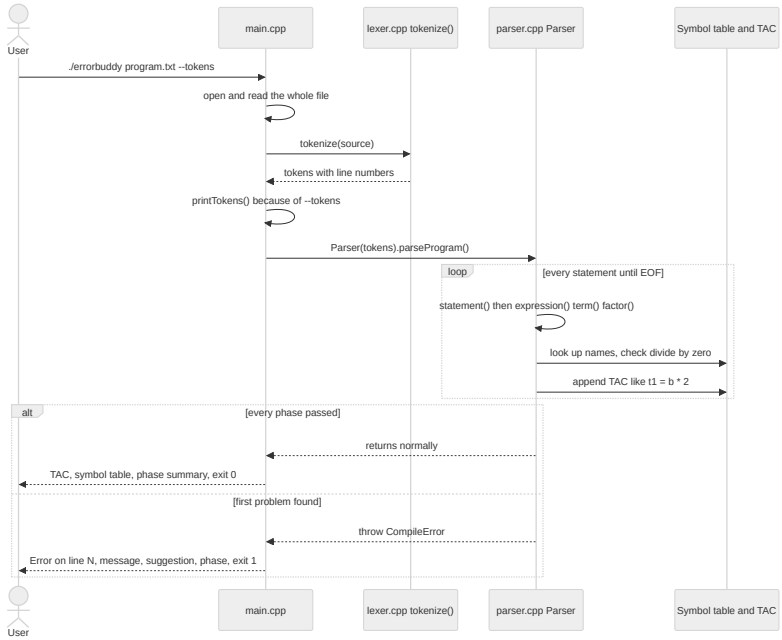
ITEM	WHAT IT DOES
`TokenType`, `Token`	13 token kinds: `KW_INT`, `KW_PRINT`, `IDENTIFIER`, `NUMBER`, the four operators, `ASSIGN`, the parentheses, `SEMICOLON` and `END_OF_FILE`. Each token stores its `type`, `text` and `line`.
`CompileError`	`{ line, phase, message, suggestion }`. Defined here so every phase can throw it.
`tokenize(src)`	A hand-written scanner with one loop over the characters: newline → `line++`; whitespace and `//` comments are skipped; runs of digits become `NUMBER`; letters or `_` start a name (`int` and `print` become keywords); single characters become symbols. It always appends `END_OF_FILE`.
Lexical errors	Lexical an unknown symbol such as `@`, a number above `INT_MAX`, or a name that starts with a digit such as `2nd`.
`tokenTypeName()`	Readable type names (`KEYWORD`, `OPERATOR`, ...) for `--tokens`.

parser.h / parser.cpp: Phases 2, 3 and 4 in one pass

ITEM	WHAT IT DOES
`Parser(tokens)`	Stores the tokens and collects every identifier into `userNames`, so temporaries never clash with real names.
`parseProgram()`, `statement()`	Rejects an empty file, then dispatches on the first token: `int` → `declaration()`, `print` → `printStatement()`, a name followed by `=` → `assignment()`.
`expression()` / `term()` / `factor()`	One function per grammar rule. `term()` handles `* /` one level deeper than `+ -`, which gives precedence. The `while` loops make operators left-associative. `factor()` recurses on `(`, up to a depth of 200.
`symbolTable`	`map<string, Symbol{declaredLine, value}>`. A name is added only after its initial value is parsed, so `int a = a + 1;` is caught.
`binary()`	Computes the value (possible because there is no input and no loops), checks for division by zero and `int` overflow, then emits `tN = left op right`.
`newTemp()`, `tac`	Temporaries are `t1, t2…`, skipping any name in `userNames`. TAC is appended in source order.
Diagnosis helpers	`expectSemicolon()`, `factorError()`, `unknownStatementError()`, `editDistance()`, `looksLike()`, `similarVariable()`. See page 6.

DIAGRAM 3 · 3_CLI_SEQUENCE.MMD

Execution flow of one compile



1. **Start.** `./errorbuddy examples/precedence.txt --tokens`. `main()` reads the arguments, opens the file and reads it into one string.
2. **Lex.** `tokenize()` returns 22 tokens plus `END_OF_FILE`, each tagged with its line. `--tokens` prints them.
3. **Parse, check and generate together.** For `int c = a + b * 2;`: `declaration()` → `expression()` → `term()` returns `a`; `+` is seen; the next `term()` takes `b`, sees `*`, takes `2` and calls `binary()`, which emits `t1 = b * 2`. Back in `expression()`, `binary()` emits `t2 = a + t1`. `expectSemicolon()` finds `;`, `c` goes into the symbol table and `c = t2` is emitted.
4. **Finish.** At `END_OF_FILE`, `main()` prints the result and exits with 0.

```
Intermediate Code (Three-Address Code):
    a = 5
    b = 10
    t1 = b * 2
    t2 = a + t1
    c = t2
    print c
```

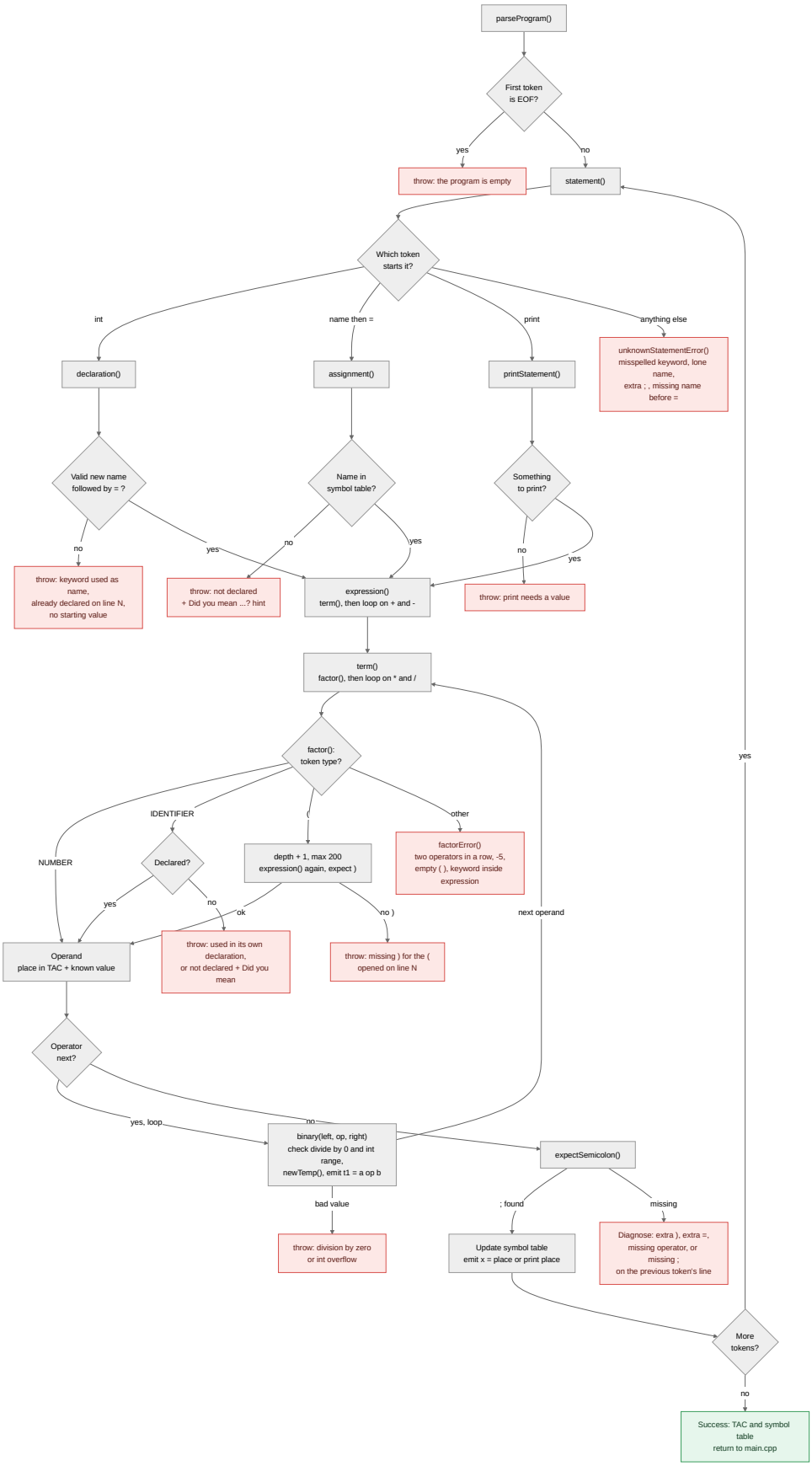
```
Error on line 3:
Division by zero. Variable 'zero' is 0
at this point, so dividing by it is
impossible.
Suggestion: Make sure the value on the
right of '/' is not zero.
(Found during: Semantic Analysis)
```

Left: TAC for `examples/precedence.txt`. Right: `examples/division_by_zero.txt`, where `zero = a - 10` is known to be 0 at compile time.

EXIT CODE	MEANING
`0`	Compiled. TAC, symbol table and phase summary printed.
`1`	Compile error. One explanation printed, no TAC.
`2`	Bad command line or file not found.
`3`	Internal compiler error (should never happen).

DIAGRAM 2 · 2_PARSER_INTERNALS.MMD

Inside the parser: decisions and where errors are thrown



PHASE 5

The error engine

Every error is thrown as one `CompileError` and caught in one place, so a run stops at the first problem. Instead of a generic “syntax error”, the throwing code looks at the tokens around the problem to name the likely cause, and reports the line where the fix belongs.

THROWN BY	PHASE	SITUATIONS IT RECOGNISES
`tokenize()`	Lexical	Unknown symbol; number too large for `int`; name starting with a digit.
`parseProgram()`	Syntax	Empty program.
`declaration()`	Syntax Semantic	Keyword used as a name; missing name; no starting value; missing `=`; already declared on line N.
`assignment()`	Semantic	Variable not declared, with a *Did you mean* hint when a similar name exists.
`printStatement()`	Syntax	`print;` with nothing to print.
`factor()`	Syntax Semantic	Undeclared name (with hint); name used in its own declaration; missing `)` for the `(` on line N; nesting deeper than 200.
`factorError()`	Syntax	Two operators in a row; `-5` (no unary minus); empty `()`; an operator with no right side; a keyword inside an expression; a stray `=`; an unexpected end of file.
`binary()`	Semantic	Division by zero (by a literal, a variable or an expression); result outside the `int` range.
`expectSemicolon()`	Syntax	Extra `)`; a second `=`; a missing operator between two values on the same line; otherwise *Expected ';'*, reported on the **previous token's line**.
`unknownStatementError()`	Syntax	Misspelled or capitalised keyword (`prnt`, `Int`); a lone name; an extra `;`; `=` with no name; a statement starting with a symbol.

Typo hints (edit distance)

`editDistance()` computes Levenshtein distance using one rolling row. `looksLike()` allows 1 edit for targets of up to 3 letters and 2 edits for longer ones. `similarVariable()` searches the symbol table and skips names shorter than 3 letters. Keyword hints use the same test against `int` and `print`.

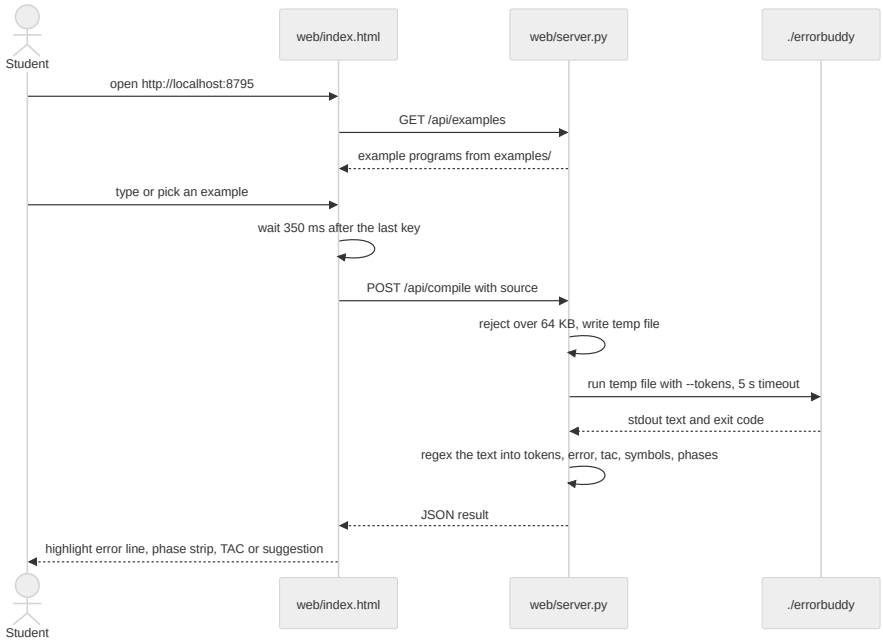
```
scroe = 2;  →  Did you mean 'score'?
prnt x;     →  Did you mean the keyword
'print'?
```

Why “previous token's line” matters

A parser only notices a missing `;` when it reaches the *next* token, which is often on the next line. `expectSemicolon()` reports `previous().line` instead, so the student is sent to the line they need to edit. In the case study this moved the exact-line score from 21/24 to 24/24.

DIAGRAM 4 · 4_WEB_SEQUENCE.MMD

Web playground, tests and evaluation



web/server.py

ROUTE	BEHAVIOUR
`GET /`	Serves `index.html`.
`GET /api/examples`	Every `examples/*.txt` except `empty.txt`.
`POST /api/compile`	Rejects input over 64 KB, writes a temp file, runs `./errorbuddy --tokens` with a 5 s timeout, and uses regexes to turn stdout into JSON: `tokens`, `error`, `tac`, `symbols`, `phases`.

The server only listens on `127.0.0.1` and builds the binary on first start if it is missing. It never reimplements compiler logic; the browser always shows the real compiler's output.

make test

Runs every `examples/*.txt` and compares stdout with `examples/expected/*.out` using `diff`. The 10 programs cover a valid program, precedence, an undeclared variable, a duplicate declaration, a missing semicolon, an invalid expression, missing and extra parentheses, division by zero and an empty file.

case_study/

`corpus.py` holds 24 hand-labelled mistakes, each written in ErrorBuddy and as equivalent C. `run_study.py` builds the shipped compiler and a variant that reports the detection-point line, and runs gcc 11.4 (`-Wall -Wextra`) on the C versions. `make_case_study.py` renders the PDF.

REPORTER	CAUGHT	EXACT LINE
gcc 11.4, first diagnostic	23/24	20/24
ErrorBuddy, detection point	24/24	21/24
ErrorBuddy, shipped	24/24	24/24

APPENDIX

Mermaid source: full architecture

Also in `~/ErrorBuddy/architecture/1_full_architecture.mmd`. The parser and sequence diagrams are in the same folder. The `classDef` lines only add colour and can be deleted if a tool ignores them.

```
flowchart TD
    USER["Student / user"]
    BUILD["Makefile: make<br>g++ -std=c++17 -Wall -Wextra -
O2<br>main.cpp + lexer.cpp + parser.cpp → ./errorbuddy"]

    subgraph ENTRY["Ways to run the compiler"]
        E_CLI["Terminal<br>./errorbuddy program.txt --
tokens<br>or make run FILE=..."]
        E_TEST["make test<br>runs every examples/*.txt"]
        E_WEB["web/index.html in the browser<br>recompiles 350 ms
after typing"]
        E_SRV["web/server.py on :8795<br>POST /api/compile: write
temp file,<br>run ./errorbuddy --tokens, 5 s timeout"]
        E_STUDY["case_study/run_study.py<br>24 labelled mistakes
from corpus.py,<br>same mistakes also sent to gcc"]
    end

    subgraph DRIVER["main.cpp - start"]
        D1["main(argc, argv)<br>read file path and --tokens
flag"]
        D2{"File given<br>and opened?"}
        D3["Print usage or cannot-open message<br>exit code 2"]
        D4["Read the whole file into one string<br>enter the try
block"]
    end

    subgraph LEXER["lexer.h + lexer.cpp - Phase 1: Lexical
Analysis"]
        L1["tokenize(source)<br>scan one character at a time"]
        L2["Skip spaces and // comments<br>count newlines to know
the line"]
        L3["Group characters into tokens:<br>NUMBER, IDENTIFIER,
keywords int and print,<br>symbols + - * / = ( ) ;"]
        L4["Token list: type, text, line<br>ends with
END_OF_FILE"]
    end

    subgraph PARSER["parser.h + parser.cpp - Phases 2, 3 and 4 in
one pass"]
        P1["Parser(tokens)<br>remember every user name"]
        P2["parseProgram()<br>reject empty file, then statement()
until EOF"]
        P3["Phase 2 Syntax: statement()<br>declaration(),
assignment(), printStatement()"]
        P4["Phase 2 Syntax: expression() → term() → factor()
<br>recursive descent, * / bind tighter than + -"]
        P5[("Phase 3 Semantic: symbol table<br>map name →
declared line, current value")]
        P6["Phase 4 TAC: binary(left, op, right)<br>compute
value, newTemp(), emit t1 = a op b"]
        P7[("TAC list<br>three-address code in source order")]
    end

    subgraph ERRORS["Phase 5: ErrorBuddy error engine (parser.cpp
+ lexer.h)"]
        X1["Typo hints<br>editDistance(), looksLike(),
similarVariable()<br>Did you mean ...?"]
        X2["Cause diagnosis<br>expectSemicolon(), factorError(),
<br>unknownStatementError()"]
        X3["CompileError thrown<br>line, phase, message,
suggestion"]
    end

    subgraph OUTPUT["main.cpp - results"]
        D5["printTokens()<br>token table, only with --tokens"]
        D6["Success: Compilation successful<br>TAC, symbol table,
phase summary<br>exit code 0"]
        D7["catch CompileError<br>Error on line N, message,
Suggestion, phase<br>no TAC shown, exit code 1"]
        D8["stdout text + exit code"]
    end

    subgraph READERS["Who reads the output"]
        R1["Terminal<br>student reads the result"]
        R2["make test<br>diff with
examples/expected/*.out<br>PASS or FAIL"]
        R3["server.py turns the text into JSON<br>index.html
highlights the error line,<br>phase strip, TAC and symbols"]
        R4["run_study.py writes out_*.txt<br>make_case_study.py
builds the case study PDF"]
    end

    USER ⟶|make| BUILD
    USER ⟶ E_CLI
    USER ⟶ E_TEST
    USER ⟶ E_WEB
    USER ⟶ E_STUDY
    E_WEB ⟶|source as JSON| E_SRV
    BUILD ⟶|binary| D1
    E_CLI ⟶ D1
    E_TEST ⟶ D1
    E_SRV ⟶ D1
    E_STUDY ⟶ D1

    D1 ⟶ D2
    D2 ⟶|no| D3
    D2 ⟶|yes| D4
    D4 ⟶|"step 1: source string"| L1
    L1 ⟶ L2
    L2 ⟶ L3
    L3 ⟶ L4
    L4 ⟶|"step 2: tokens"| P1
    P1 ⟶ P2
    P2 ⟶ P3
    P3 ⟶ P4
    P4 ⟶|"look up and add names"| P5
    P4 ⟶ P6
    P6 ⟶ P7
    P3 ⟶|"emit x = t1 or print t2"| P7
    P7 ⟶|"step 3: every statement passed"| D6
    L4 -.→|"--tokens"| D5

    L3 -.→|"unknown symbol, 2nd, number too big"| X3
    P3 -.→|"missing ; or bad start"| X2
    P4 -.→|"missing value or ), undeclared name"| X2
    P6 -.→|"division by zero, int overflow"| X3
    P5 ⟶|declared names| X1
    X1 ⟶|suggestion| X2
    X2 ⟶|throw| X3
    X3 -.→|"caught once in main"| D7

    D3 ⟶ D8
    D5 ⟶ D8
    D6 ⟶ D8
    D7 ⟶ D8
    D8 ⟶ R1
    D8 ⟶ R2
    D8 ⟶ R3
    D8 ⟶ R4

    classDef err fill:#fde8e6,stroke:#d9534f,color:#5a1a14
    classDef ok fill:#e6f6ec,stroke:#3a9d5d,color:#123d22
    classDef store fill:#eef2fb,stroke:#5b7bd5,color:#1b2b55
    class X1,X2,X3,D7,D3 err
    class D6 ok
    class P5,P7 store
```